

Creación y Gestión de Usuarios con Prisma y Node.js

Resumen

La creación de una API robusta requiere no solo una conexión a la base de datos, sino también la capacidad de gestionar y mostrar información de manera eficiente. En esta fase del desarrollo, aprenderemos a poblar nuestra base de datos, actualizar modelos y preparar nuestra aplicación para implementar la autenticación de usuarios, elementos fundamentales para cualquier aplicación moderna.

¿Cómo poblar nuestra base de datos con información inicial?

Para comenzar a trabajar con datos reales en nuestra API, necesitamos crear información inicial o "semilla" que nos permita probar la funcionalidad. Existen dos enfoques principales para generar estos datos:

1. Consultar la documentación de Prisma para crear scripts manualmente.
2. Utilizar una inteligencia artificial para generar el script necesario.

En este caso, optamos por la segunda opción, creando un archivo llamado `seed.js` que contiene la lógica para poblar nuestra base de datos:

```
// Utilizamos el cliente de Prisma para interactuar con la base de datos
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function main() {
  // Creación de usuarios de demostración
  const users = [
    { name: 'Usuario 1', email: 'usuario1@ejemplo.com' },
    { name: 'Usuario 2', email: 'usuario2@ejemplo.com' },
    { name: 'Usuario 3', email: 'usuario3@ejemplo.com' }
  ];

  for (const user of users) {
    await prisma.user.create({
      data: user
    });
  }

  console.log('Usuarios de demostración creados con éxito');
}
```

```
main()
  .catch(e => console.error(e))
  .finally(async () => await prisma.$disconnect());
```

Este script utiliza el cliente de Prisma para crear usuarios de ejemplo en nuestra base de datos. Para ejecutarlo, simplemente usamos el comando:

```
node prisma/seed.js
```

Una vez ejecutado, podemos verificar que los datos se han creado correctamente accediendo al endpoint db-user de nuestra API, donde ahora deberíamos ver la información de los usuarios creados.

¿Cómo actualizar nuestro modelo de usuario para soportar autenticación?

Para implementar la autenticación en nuestra aplicación de citas médicas, necesitamos actualizar el modelo de usuario con campos adicionales:

Añadiendo campos necesarios

Nuestro modelo de usuario debe incluir:

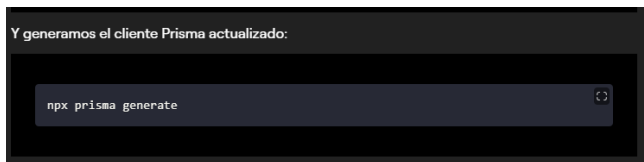
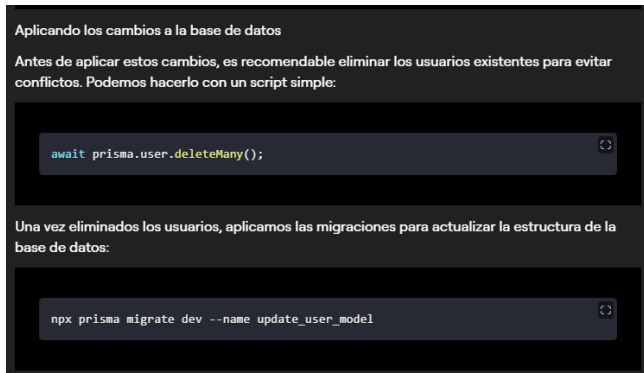
1. **Password:** Para almacenar la contraseña del usuario.
2. **Rol:** Para definir los permisos del usuario (admin o user).
3. **Appointments:** Para relacionar al usuario con sus citas médicas.

La estructura actualizada en nuestro archivo schema.prisma quedaría así:

```
La estructura actualizada en nuestro archivo schema.prisma quedaría así:

model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String
  password String
  role    Role
  appointments Appointment[]
}

enum Role {
  ADMIN
  USER
}
```



Estos comandos sincronizarán nuestra base de datos con el nuevo modelo y actualizarán el cliente Prisma para que podamos utilizar los nuevos campos en nuestro código.

¿Qué pasos seguir para implementar la autenticación?

Con nuestro modelo de usuario actualizado, estamos listos para implementar la autenticación en nuestra aplicación. Los próximos pasos incluirán:

1. **Crear endpoints para registro de usuarios:** Permitir a los usuarios crear cuentas con email, contraseña y asignarles un rol.
2. **Implementar la autenticación:** Verificar credenciales y generar tokens de acceso.
3. **Proteger rutas:** Asegurar que solo usuarios autenticados puedan acceder a ciertas funcionalidades.
4. **Gestionar permisos basados en roles:** Diferenciar entre acciones permitidas para administradores y usuarios regulares.

Es importante mantener buenas prácticas de seguridad, como el hash de contraseñas y la validación de datos de entrada, para proteger la información de nuestros usuarios.

La estructura que estamos construyendo nos permitirá no solo autenticar usuarios, sino también implementar la lógica de negocio específica para nuestra aplicación de citas médicas, como listar horas disponibles y crear nuevas citas.

El desarrollo de una API robusta requiere atención a estos detalles fundamentales que estamos abordando paso a paso. ¿Has trabajado con scripts de semilla antes? Comparte tus experiencias o dudas en la sección de comentarios para enriquecer el aprendizaje de toda la comunidad.